

Relatório 4 - Prática: Principais Bibliotecas e Ferramentas Python para Aprendizado de Máquina (I)

Gustavo Nunes da Silva

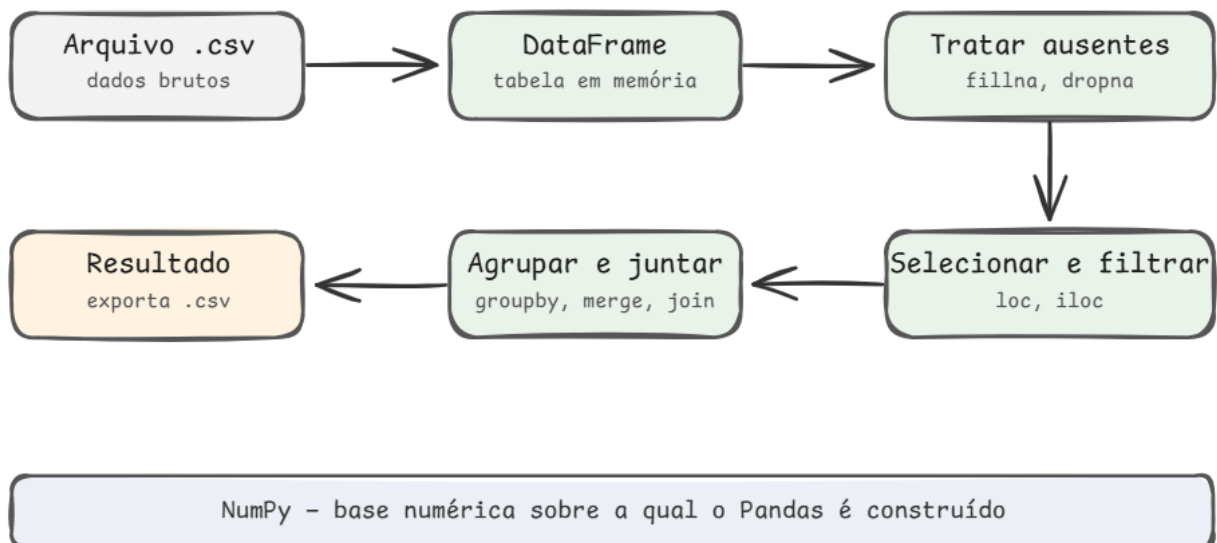
1. Introdução

O objetivo deste card foi entender mais na prática as ferramentas que são usadas no dia a dia de aprendizado de máquina o Jupyter Notebook como ambiente de trabalho e as bibliotecas NumPy e Pandas para trabalhar com os dados. A tarefa foi instalar e configurar esse ambiente e passar pelas seções do curso [1] praticando a criação e o tratamento de estruturas de dados até chegar na leitura de arquivos externos

2. Desenvolvimento

Eu montei essa primeira imagem porque no começo as três ferramentas pareciam assuntos separados, mas pra mim ficou mais fácil entender quando desenhei onde cada uma entra. O Jupyter é o ambiente onde tudo roda o NumPy fica responsável pela parte dos cálculos e o Pandas trabalha por cima dele cuidando dos dados em formato de tabela até chegar na análise

Jupyter Notebook – ambiente onde tudo roda



Fonte – Autoria própria (elaborado no excalidraw.io).

Tudo começa com o Jupyter onde eu entendi que ele não era uma biblioteca, mas sim um ambiente de desenvolvimento assim como o Colab da Google. Ele permite escrever células de código e pra mim isso facilita bastante a visualização a organização e o entendimento do código já que cada parte roda sozinha. Ele pode ser instalado e

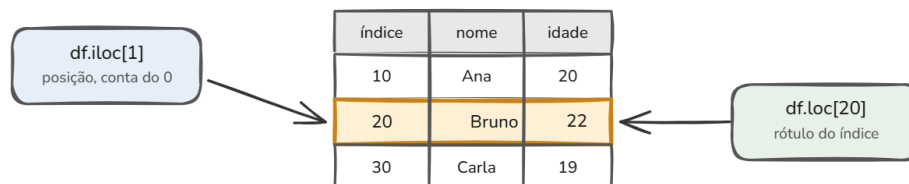
executado na própria máquina funcionando como uma espécie de IDE. Essa execução em células foi o que mais mudou pra mim em relação a um script comum porque em análise de dados eu consigo carregar um conjunto pesado uma vez e ficar testando transformações em cima dele e sem as células cada teste teria que executar o arquivo inteiro de novo.

Na configuração do ambiente eu optei por criar um ambiente virtual nativo do Python o venv dentro do ecossistema do Anaconda e isso me fez ter que adaptar parte dos comandos apresentados no curso. O motivo de usar um ambiente virtual, e essa foi uma das partes que mais me fez perder tempo até entender, é que as bibliotecas ficam isoladas por projeto. Cada projeto passa a ter suas próprias versões de Numpy e Pandas e atualizar uma biblioteca para um trabalho não quebra outro que dependia da versão anterior.

No Numpy eu trabalhei com arrays criados a partir de listas com o array ou gerados direto pelo arange e depois redimensionados pelo reshape que só reorganiza os mesmos elementos e por isso a quantidade precisa continuar batendo. O que deixa a biblioteca rápida é a base escrita em C que executa as operações sobre o array inteiro sem precisar de laço. A parte que mais mudou meu jeito de escrever código foi o fatiamento porque ele não cria um array novo e devolve um apontamento para o original então se eu alterar o recorte eu também altero o dado de origem e só o copy separa os dois. Junto disso eu vi que a divisão por zero não interrompe a execução e devolve inf ou NaN dentro do array. Os dois casos me pareceram perigosos pelo mesmo motivo por que são silenciosos e o problema só aparece mais pra frente quando uma média volta como NaN sem eu entender de onde veio.

No Pandas eu vi que as estruturas são a Series que é unidimensional e parecida com um dicionário que permite operações por índice e o Dataframe que é bidimensional e em que cada coluna é uma Series. A seleção pode ser feita de duas formas que parecem iguais, mas não são o iloc navega pela posição numérica contando do zero e o loc navega pelo rótulo do índice. Eu montei a essa segunda imagem justamente porque esse foi o erro que mais cometi no começo e ele é difícil de perceber já que trocar um pelo outro não dá nenhuma exceção apenas devolve a linha errada

o mesmo Dataframe dois caminhos até a mesma linha



`df.loc[1]` levanta `KeyError` não existe rótulo 1
se o índice ele fosse 0, 1, 2 os dois apontariam a mesma linha
e num índice fora de ordem devolveriam linhas diferentes, sem erro

Fonte – Autoria própria (elaborado no excalidraw.io).

Ainda na manipulação da tabela eu vi que o drop remove linhas ou colunas, mas tem uma pegadinha no parâmetro inplace por padrão ele vem como falso então o método devolve uma cópia já alterada e deixa o Dataframe original intacto. Na prática eu posso executar a remoção ver o resultado certo na tela e continuar achando que apaguei a coluna quando na verdade não apaguei nada. Percebi que isso segue o mesmo raciocínio do fatiamento do NumPy nos dois casos eu preciso saber se a operação está mexendo no dado original ou se só está devolvendo uma versão modificada dele

No tratamento de dados ausentes eu vi que o fillna preenche os NaN com um valor definido e o dropna elimina linhas ou colunas conforme o axis escolhido. Entendi que a escolha entre os dois não é só uma questão técnica e sim de contexto porque descartar diminui a base e preencher coloca um valor que ninguém mediu. Eu usaria o dropna quando os ausentes fossem poucos diante do total ou quando estivessem em uma coluna que não é importante para a análise e usaria o fillna quando descartar fizesse eu perder boa parte da base preenchendo com a média ou a mediana da própria coluna para não mudar muito a distribuição

Por fim eu cheguei nas operações que transformam a tabela em resposta e a principal delas é o groupby. Ele reúne as linhas que têm o mesmo valor em uma coluna e aplica a operação dentro de cada grupo devolvendo uma linha por grupo em vez do registro original. Em uma planilha de vendas lançada venda a venda se eu agrupar por vendedor e somar a quantidade eu consigo o total de cada um e isso é exatamente a resposta que eu quero e não está em nenhuma linha isolada da tabela. Quando o dado está espalhado em mais de uma tabela o merge faz a junção pelas colunas em comum com as variações inner left right e outer definindo o que acontece com as chaves que não têm par. Fechando o fluxo da Figura 1 eu li e gravei arquivos .csv que é o formato em que os dados normalmente chegam e vi que o Pandas abre planilhas de Excel e arquivos de texto pela mesma família de métodos então não preciso converter a base antes de começar

3. Conclusão

A prática deixou mais claro para mim que o trabalho com aprendizado de máquina começa muito antes do modelo na parte de preparar e entender os dados e que essas três ferramentas cobrem exatamente esse trecho. O Jupyter dá o ambiente interativo que me permite testar transformações sem executar tudo de novo o NumPy cuida da performance das operações numéricas por ter base em C e o Pandas oferece a estrutura em tabela em que os dados realmente chegam. O que mais me marcou foi perceber que os dois erros mais fáceis de cometer são silenciosos o fatiamento que aponta para o original em vez de copiar e o drop que sem o inplace não altera nada. Saio do card

entendendo que dominar essas bibliotecas é menos sobre decorar os métodos e mais sobre saber o que cada operação faz com o dado original

4. Referências

TADEWALD, Rodrigo Soares; PIERIAN DATA INTERNATIONAL. Python para Data Science e Machine Learning – COMPLETO. Udemy, 2019.